MỤC LỤC

I. THÔNG TIN THÀNH VIÊN.....4

II. ĐÁNH GIÁ MỨC ĐỘ HOÀN THÀNH.....4

III. BẢNG PHÂN CÔNG CÔNG VIỆC TRONG ĐỒ ÁN.....5

IV. CÀI ĐẶT GIAO THỨC RTSP PHÍA CLIENT (IMPLEMENT RTSP PROTOCOL IN CLIENT).....6

1. Tổng quan về Máy trạng thái RTSP (RTSP State Machine).....6

2. Hiện thực hóa trong mã nguồn (Implementation Details).....6

V. HƯỚNG ĐÓNG GÓI GÓI TIN RTP PHÍA SERVER (RTP PACKETIZATION IN SERVER).....7

1. Cấu trúc gói tin RTP (RTP Header Structure).....7

2. Hiện thực hóa đóng gói Bit (Bit Manipulation).....8

3. Quy trình gởi gói tin tại Server.....9

VI. KẾT LUẬN PHẦN CÀI ĐẶT (IMPLEMENT THE RTPS PROTOCOL IN THE CLIENT AND IMPLEMENT THE RTP PACKETIZATION IN THE SERVER).....9

VII. HD VIDEO STREAMING.....9

1. Maximum Transmission Unit (MTU) và Fragmentation.....9

2. Fragment Header Design.....10

3. Network Statistics Metrics.....11

4. Chi tiết về Project.....11

VIII. CLIENT-SIDE CACHING.....22

1. Đặt vấn đề.....22

2. Quy trình xử lí (Logic Flow).....22

3. Kết quả đạt được.....24

4. Kết luận và hướng phát triển.....24

IX. TÀI LIỆU THAM KHẢO.....24

THÔNG TIN CHUNG VỀ ĐỒ ÁN

I. THÔNG TIN THÀNH VIÊN

MÃ SỐ SINH VIÊN	HỌ VÀ TÊN
24120284	Trần Tiến Đạt
24120245	Trần Lê Đức Việt
24120239	Phí Công Tuấn

II. ĐÁNH GIÁ MỨC ĐỘ HOÀN THÀNH

STT	Hạng mục (Feature)	Yêu cầu kỹ thuật (Technical Requirements)	Mức độ hoàn thành	Kết quả đạt được (Evidence)
1	**Implement RTSP & RTP**	- Client điều khiển được trạng thái Server. - Gói tin RTP đúng cấu trúc 12-byte Header.	100%	- Client gửi đúng Sequence Number và Session ID. - Server đóng gói payload và gửi thành công qua UDP.
2	**HD Video Streaming**	- Hỗ trợ Video độ phân giải cao/File chuẩn MJPEG. - Không bị lỗi ValueError khi đọc frame.	100%	- Đã đọc thành công các file .mjpeg (chuẩn MJPEG) thay vì định dạng custom 5-byte. - Video hiển thị rõ nét, frame rate ổn định.
3	**Client-Side Caching**	- Sử dụng bộ nhớ đệm cục bộ (Local Cache) để xử lý frame. - Hiển thị ảnh từ Cache.	100%	- Hệ thống tạo file cache-[session].jpg liên tục trong quá trình chạy. - Hình ảnh hiển thị lên GUI được load từ file cache này, giúp tách biệt luồng nhận

				mạng và luồng hiển thị.
Tổng	**ĐỒ ÁN MÔN HỌC**	Hoàn thành 1 Core + 2 Optionals	100%	Hệ thống hoạt động hoàn chỉnh.

III. BẢNG PHÂN CÔNG CÔNG VIỆC TRONG ĐỒ ÁN

Bảng phân công công việc trong đồ án:

STT	Thành viên	Nhiệm vụ chính	Chi tiết công việc thực hiện
1	**Trần Lê Đức Việt** (Core Protocols)	**Implement RTSP Protocol & RTP Packetization** (Nhiệm vụ cơ bản)	- **RTSP (Client):** Xây dựng máy trạng thái (INIT, READY, PLAYING) và hàm gửi request (SETUP, PLAY, PAUSE, TEARDOWN). - **RTP (Server):** Xây dựng class RtpPacket, viết hàm encode() sử dụng thao tác bit (bitwise) để đóng gói Header 12-byte chuẩn. - Thiết lập kết nối Socket TCP (cho RTSP) và UDP (cho RTP).
2	**Trần Tiến Đạt** (Advanced Media)	**HD Video Streaming** (Nâng cao 1)	- **Xử lý file Video lớn:** Viết lại class VideoStream.py để tự động tìm marker JPEG (0xFFD8, 0xFFD9) thay vì dùng header cố định, giúp đọc được file HD dung lượng lớn. - **Phân mảnh (Fragmentation):** Xử lý phía Server để chia nhỏ frame lớn thành các gói tin nhỏ hơn kích thước MTU (Max Transmission Unit) để gửi qua UDP không bị lỗi.
3	**Phí Công Tuấn** (Optimazation)	**Client – Side Caching** (Nâng cao 2)	- **Cơ chế Caching:** Hiện thực hóa hàm writeFrame trong Client.py để lưu gói tin nhận được xuống ổ cứng (cache.jpg) trước khi hiển thị. - **GUI & Sync:** Đảm bảo việc đọc/ghi Cache không làm đơ giao diện người dùng. - **Dọn dẹp:** Xử lý xóa file Cache khi

			kết thúc phiên (teardown).

NỘI DUNG ĐỒ ÁN

IV. CÀI ĐẶT GIAO THỨC RTSP PHÍA CLIENT (IMPLEMENT RTSP PROTOCOL IN CLIENT)

1. Tổng quan về Máy trạng thái RTSP (RTSP State Machine)

- Trong mô hình Video Streaming, Client đóng vai trò điều khiển luồng dữ liệu thông qua giao thức RTSP (Real-Time Streaming Protocol). Client hoạt động dựa trên một máy trạng thái (State Machine) với các trạng thái chính: **INIT**, **READY**, và **PLAYING**.
- Hành vi của Client được xác định bởi các yêu cầu (Requests) gửi đến Server:
    - **SETUP:** Thiết lập kết nối, chuyển từ trạng thái INIT sang READY.
    - **PLAY:** Bắt đầu nhận dữ liệu video, chuyển từ READY sang PLAYING.
    - **PAUSE:** Tạm dừng luồng video, chuyển từ PLAYING về READY.
    - **TEARDOWN:** Kết thúc phiên làm việc, đóng kết nối và quay về INIT.

2. Hiện thực hóa trong mã nguồn (Implementation Details)

- Trong class Client (file Client.py), chúng tôi quản lý trạng thái thông qua biến self.state và gửi yêu cầu thông qua hàm sendRtspRequest.
- **Cấu trúc bản tin RTSP:** Mỗi bản tin RTSP gửi đi là một chuỗi văn bản (String) tuân thủ định dạng chuẩn, bao gồm: Method, Filename, RTSP Version, CSeq (Sequence Number), và các trường header khác (Transport, Session).
- Đoạn mã xử lý gửi yêu cầu trong **Client.py** được cài đặt như sau:

```python
def sendRtspRequest(self, requestCode):
    """Send RTSP request to the server."""

    # Setup request
    if requestCode == self.SETUP and self.state == self.INIT:
        threading.Thread(target=self.recvRtspReply).start()
        # Update RTSP sequence number.
        self.rtspSeq += 1

        # Write the RTSP request to be sent.
        request = f"SETUP {self.fileName} RTSP/1.0\nCSeq: {self.rtspSeq}\nTransport: RTP/UDP; client_port= {self.rtpPort}"

        # Keep track of the sent request.
        self.requestSent = self.SETUP

    # Play request
    elif requestCode == self.PLAY and self.state == self.READY:
        # Update RTSP sequence number.
        self.rtspSeq += 1

        # Write the RTSP request to be sent.
        request = f"PLAY {self.fileName} RTSP/1.0\nCSeq: {self.rtspSeq}\nSession: {self.sessionId}"

        # Keep track of the sent request.
        self.requestSent = self.PLAY

    # Pause request
    elif requestCode == self.PAUSE and self.state == self.PLAYING:
        # Update RTSP sequence number.
        self.rtspSeq += 1

        # Write the RTSP request to be sent.
        request = f"PAUSE {self.fileName} RTSP/1.0\nCSeq: {self.rtspSeq}\nSession: {self.sessionId}"

        # Keep track of the sent request.
        self.requestSent = self.PAUSE

    # Teardown request
    elif requestCode == self.TEARDOWN and not self.state == self.INIT:
        # Update RTSP sequence number.
        self.rtspSeq += 1

        # Write the RTSP request to be sent.
        request = f"TEARDOWN {self.fileName} RTSP/1.0\nCSeq: {self.rtspSeq}\nSession: {self.sessionId}"

        # Keep track of the sent request.
        self.requestSent = self.TEARDOWN
    else:
        return

    # Send the RTSP request using rtspSocket.
    self.rtspSocket.send(request.encode())

    print('\nData sent:\n' + request)
```

- Giải thích kỹ thuật:
    - **CSeq (Sequence Number):** Được tăng lên 1 sau mỗi lần gửi request để đảm bảo tính đồng bộ giữa Client và Server.
    - **Session ID:** Sau khi nhận phản hồi SETUP từ Server, Client lưu lại Session ID và gửi kèm trong các lệnh PLAY, PAUSE, TEARDOWN để Server nhận diện phiên làm việc.

V. HƯỚNG ĐÓNG GÓI GÓI TIN RTP PHÍA SERVER (RTP PACKETIZATION IN SERVER)

1. Cấu trúc gói tin RTP (RTP Header Structure)

- Dữ liệu Video không được gửi đi dưới dạng thô (raw data) mà được đóng gói vào các gói tin RTP (Real-time Transport Protocol) để truyền qua giao thức UDP. Theo tài liệu lý thuyết, mỗi gói tin RTP bao gồm một Header cố định kích thước 12 bytes, theo sau là phần dữ liệu (Payload).
- Các trường quan trọng trong RTP Header bao gồm:
    - **Version (V):** 2 bits (thường là 2).
    - **Padding (P):** 1 bit.
    - **Extension (X):** 1 bit.
    - **CSRC Count (CC):** 4 bits.
    - **Marker (M):** 1 bit (đánh dấu kết thúc frame).
    - **Payload Type (PT):** 7 bits (định dạng MJPEG là 26).
    - **Sequence Number:** 16 bits (số thứ tự gói tin).
    - **Timestamp:** 32 bits (thời gian lấy mẫu).
    - **SSRC:** 32 bits (định danh nguồn đồng bộ).

2. Hiện thực hóa đóng gói Bit (Bit Manipulation)

- Việc tạo RTP Header đòi hỏi kỹ thuật xử lý bit (Bit Manipulation) chính xác để nhồi nhét nhiều trường thông tin vào đúng số byte quy định. Tôi đã cài đặt logic này trong class RtpPacket (file **RtpPacket.py**).
- Dưới đây là phương thức **encode** thực hiện việc đóng gói:

```python
def encode(self, version, padding, extension, cc, seqnum, marker, pt, ssrc, payload,
        fragment_id=0, total_fragments=1, fragment_index=0):
    """Encode the RTP packet with header fields and payload."""
    timestamp = int(time())
    self.header = bytearray(HEADER_SIZE)

    # Fill the header bytearray with RTP header fields
    # Byte 0: V(2), P(1), X(1), CC(4)
    self.header[0] = (version << 6) | (padding << 5) | (extension << 4) | cc

    # Byte 1: M(1), PT(7)
    self.header[1] = (marker << 7) | pt

    # Bytes 2-3: Sequence number
    self.header[2] = (seqnum >> 8) & 0xFF
    self.header[3] = seqnum & 0xFF

    # Bytes 4-7: Timestamp
    self.header[4] = (timestamp >> 24) & 0xFF
    self.header[5] = (timestamp >> 16) & 0xFF
    self.header[6] = (timestamp >> 8) & 0xFF
    self.header[7] = timestamp & 0xFF

    # Bytes 8-11: SSRC
    self.header[8] = (ssrc >> 24) & 0xFF
    self.header[9] = (ssrc >> 16) & 0xFF
    self.header[10] = (ssrc >> 8) & 0xFF
    self.header[11] = ssrc & 0xFF

    # Add fragmentation header if this is a fragmented packet
    if total_fragments > 1:
        self.fragmentHeader = bytearray(FRAGMENT_HEADER_SIZE)
        self.fragmentHeader[0] = (fragment_id >> 8) & 0xFF
        self.fragmentHeader[1] = fragment_id & 0xFF
        self.fragmentHeader[2] = total_fragments & 0xFF
        self.fragmentHeader[3] = fragment_index & 0xFF
    else:
        self.fragmentHeader = None

    # Get the payload from the argument
    self.payload = payload
```

3. Quy trình gởi gói tin tại Server

- Tại phía Server (file ServerWorker.py), quá trình đóng gói và gửi đi diễn ra như sau:
    - **B1**. Đọc một khung hình (frame) từ VideoStream.
    - **B2**. Tạo đối tượng RtpPacket và gọi hàm encode với các thông số chuẩn.
    - **B3**. Gửi gói tin qua socket UDP.
- Đoạn mã thực hiện trong **ServerWorker.py**:

```python
def makeRtp(self, payload, frameNbr, fragment_id=0, total_fragments=1, fragment_index=0):
    """RTP-packetize the video data with optional fragmentation support."""
    version = 2
    padding = 0
    extension = 0
    cc = 0
    marker = 1 if (fragment_index == total_fragments - 1) else 0  # Set marker on last fragment
    pt = 26 # MJPEG type
    seqnum = frameNbr
    ssrc = 0

    rtpPacket = RtpPacket()

    rtpPacket.encode(version, padding, extension, cc, seqnum, marker, pt, ssrc, payload,
                     fragment_id, total_fragments, fragment_index)

    return rtpPacket.getPacket()
```

VI. KẾT LUẬN PHẦN CÀI ĐẶT (IMPLEMENT THE RTPS PROTOCOL IN THE CLIENT AND IMPLEMENT THE RTP PACKETIZATION IN THE SERVER)

- Việc cài đặt thành công RTSP Client và RTP Packetization đảm bảo:
  - **Điều khiển luồng:** Client có thể ra lệnh cho Server bắt đầu hoặc dừng gửi dữ liệu một cách chính xác.
  - **Đóng gói dữ liệu:** Server tuân thủ chuẩn RFC của RTP, giúp dữ liệu video được phân mảnh, đánh số thứ tự và gắn nhãn thời gian, cho phép Client (hoặc các trình phát video chuẩn khác) giải mã và hiển thị đúng thứ tự khung hình.

VII. HD VIDEO STREAMING

1. Maximum Transmission Unit (MTU) và Fragmentation

a. MTU là gì ?

- MTU (Maximum Transmission Unit) là kích thước tối đa của một packet có thể truyền qua mạng mà không bị phân mảnh ở tầng IP.
  - Ethernet MTU: 1500 bytes
  - Internet path MTU: thường 1400-1500 bytes
  - Safe MTU: 1400 bytes (tránh fragmentation ở IP layer)

b. Vấn đề với HD Video

- Frame 720p (~50KB) cần gửi qua network với MTU 1400 bytes. Nếu gửi trực tiếp → IP layer fragmentation → packet loss cao → jitter.

- Giải pháp: Application-level fragmentation
    - Chia frame thành fragments nhỏ (≤ 1384 bytes)
    - Thêm fragment header để reassemble
    - Gửi từng fragment riêng biệt

c. Payload Size

- MTU = 1400 bytes
- IP header = 20 bytes
- UDP header = 8 bytes
- RTP header = 12 bytes
- Fragment header = 4 bytes

→ MAX_PAYLOAD_SIZE = 1400 - 20 - 8 - 12 - 4 = 1356 bytes (Trong implementation sử dụng 1384 = 1400 - 12 - 4 để đơn giản, giả định IP/UDP header được xử lý tự động)

2. Fragment Header Design

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|         Fragment ID           |  Total Frags  | Fragment Index|
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

- Fragment ID (2 bytes): Unique identifier cho mỗi frame được fragment
- Total Frags (1 byte): Tổng số fragments trong frame (max 255)
- Fragment Index (1 byte): Index của fragment này (0 to Total - 1)

```
self.fragmentHeader = bytearray(FRAGMENT_HEADER_SIZE)
self.fragmentHeader[0] = (fragment_id >> 8) & 0xFF
self.fragmentHeader[1] = fragment_id & 0xFF
self.fragmentHeader[2] = total_fragments & 0xFF
self.fragmentHeader[3] = fragment_index & 0xFF
```

- **Ví dụ:**

  Frame 720p = 52,480 bytes

MAX_PAYLOAD_SIZE = 1,384 bytes

Total fragments = ⌈52,480 / 1,384⌉ = 38 fragments

Fragment 0: fragment_id=100, total=38, index=0, payload=1384 bytes

Fragment 1: fragment_id=100, total=38, index=1, payload=1384 bytes

...

Fragment 37: fragment_id=100, total=38, index=37, payload=464 bytes

3. Network Statistics Metrics

- **Bandwidth** (Kbps) = (Total Bytes * 8) / (Elapsed Time * 1024): đo lượng dữ liệu truyền tải trong một đơn vị thời gian
- **Loss Rate** (%) = (Packets Lost / Total Packets Sent) * 100: đo tỷ lệ packets bị mất trong quá trình truyền
- **Latency** (ms) = Receive Time - Send Time: thời gian từ khi gửi đến khi nhận packet, ảnh hưởng đến real-time experience
- **Jitter** (ms) = |ArrivalDelta - TransitDelta|: độ dao động của latency giữa các packets. Jitter càng cao, video càng giật

4. Chi tiết về Project

a. ServerWorker.Py

- **Hàm sendRtp**

```python
def sendRtp(self):
    """Main RTP sending loop với fragmentation support."""
    while True:
        self.clientInfo['event'].wait(0.02)  # 50 FPS

        # Check pause/stop
        if self.clientInfo['event'].isSet():
            break

        # Get next frame
        data = self.clientInfo['videoStream'].nextFrame()
        if data:
            frameLength = len(data)
            curFrameNbr = self.clientInfo['videoStream'].frameNum()

            # Decide: fragment hay gửi trực tiếp?
            if frameLength > self.MAX_PAYLOAD_SIZE:
                # Frame lớn → fragment
                self.sendFragmentedFrame(data, frameLength, curFrameNbr)
            else:
                # Frame nhỏ → gửi trực tiếp
                rtpPacket = self.makeRtp(data, curFrameNbr)
                self.clientInfo['rtpSocket'].sendto(
                    rtpPacket.getPacket(),
                    (self.clientInfo['rtspSocket'][1][0],
                     self.clientInfo['rtpPort'])
                )
                self.stats.recordPacketSent(len(rtpPacket.getPacket()))
                self.stats.recordFrameSent()

            # Print statistics every 100 frames
            if curFrameNbr % 100 == 0:
                stats = self.stats.getStats()
                print(f"[Server] Frame {curFrameNbr} | "
                      f"Size: {frameLength} bytes | "
                      f"BW: {stats['bandwidth_sent_kbps']:.0f} Kbps | "
                      f"Fragments: {stats['fragments_sent']}")
```

- **Hàm sendFragmentedFrame**

```python
def sendFragmentedFrame(self, frameData, frameLength, frameNbr):
    """
    Phân mảnh và gửi frame lớn.

    Algorithm:
    1. Calculate số fragments cần thiết
    2. Increment global fragmentId
    3. For each fragment:
        - Extract payload chunk
        - Create RTP packet với fragment header
        - Send via UDP
        - Add inter-fragment delay
    """
    # Calculate total fragments
    totalFragments = (frameLength + self.MAX_PAYLOAD_SIZE - 1) // \
                     self.MAX_PAYLOAD_SIZE

    # Increment fragment ID (unique cho mỗi frame)
    self.fragmentId += 1
    if self.fragmentId > 65535:  # 16-bit limit
        self.fragmentId = 1

    print(f"[Server] Fragmenting frame {frameNbr}: "
          f"{frameLength} bytes into {totalFragments} fragments")

    # Send each fragment
    for i in range(totalFragments):
        # Extract fragment payload
        start = i * self.MAX_PAYLOAD_SIZE
        end = min(start + self.MAX_PAYLOAD_SIZE, frameLength)
        fragmentData = frameData[start:end]
```

```python
        # Create RTP packet với fragment header
        rtpPacket = self.makeRtp(
            fragmentData,
            frameNbr,
            fragment_id=self.fragmentId,
            total_fragments=totalFragments,
            fragment_index=i
        )

        # Send via UDP
        self.clientInfo['rtpSocket'].sendto(
            rtpPacket.getPacket(),
            (self.clientInfo['rtspSocket'][1][0],
             self.clientInfo['rtpPort'])
        )

        # Record statistics
        self.stats.recordPacketSent(len(rtpPacket.getPacket()))
        self.stats.recordFragmentSent()

        # Inter-fragment delay (0.1ms)
        # Prevents network congestion and packet loss
        time.sleep(0.0001)

    # Record frame sent
    self.stats.recordFrameSent()
```

b.Client.py

- **Hàm handleFragmentedPacket**

```python
def handleFragmentedPacket(self, rtpPacket):
    """
    Nhận và reassemble fragmented packets.

    Data structures:
    - fragmentBuffer: {fragment_id: {index: payload}}
    - fragmentMetadata: {fragment_id: (total, timestamp)}
    """
    # Extract fragment info
    fragmentId = rtpPacket.getFragmentId()
    totalFragments = rtpPacket.getTotalFragments()
    fragmentIndex = rtpPacket.getFragmentIndex()
    payload = rtpPacket.getPayload()

    # Initialize buffer cho frame mới
    if fragmentId not in self.fragmentBuffer:
        self.fragmentBuffer[fragmentId] = {}
        self.fragmentMetadata[fragmentId] = (totalFragments, time.time())
        print(f"[Client] New fragmented frame: ID={fragmentId}, "
              f"total={totalFragments} fragments")

    # Store fragment
    self.fragmentBuffer[fragmentId][fragmentIndex] = payload
    self.stats.recordFragmentReceived()

    # Check if complete
    if len(self.fragmentBuffer[fragmentId]) == totalFragments:
        # Reassemble in order
        frameData = b''.join([
            self.fragmentBuffer[fragmentId][i]
            for i in range(totalFragments)
        ])
```

```
            frameSize = len(frameData)
            print(f"[Client] Frame {self.frameNbr} reassembled from "
                  f"{totalFragments} fragments ({frameSize} bytes)")

            # Display frame
            self.writeFrame(frameData)
            self.stats.recordFrameReceived()
            self.frameNbr += 1

            # Cleanup
            del self.fragmentBuffer[fragmentId]
            del self.fragmentMetadata[fragmentId]

        # Timeout check (cleanup incomplete frames)
        self.cleanupFragmentBuffer()
```

- **Hàm cleanupFragmentBuffer**

```
def cleanupFragmentBuffer(self, timeout=5.0):
    """Remove incomplete frames after timeout."""
    currentTime = time.time()
    toDelete = []

    for fragmentId, (total, timestamp) in self.fragmentMetadata.items():
        if currentTime - timestamp > timeout:
            received = len(self.fragmentBuffer[fragmentId])
            print(f"[Client] Dropping incomplete frame: ID={fragmentId}, "
                  f"received {received}/{total} fragments")
            toDelete.append(fragmentId)
            self.stats.recordFrameLost()

    for fragmentId in toDelete:
        del self.fragmentBuffer[fragmentId]
        del self.fragmentMetadata[fragmentId]
```

c. VideoStream.py

- **Hàm getLastFrameSize**: trả về kích thước của frame cuối cùng

```
def getLastFrameSize(self):
    """Get the size of the last frame read."""
    return self.lastFrameSize
```

- **Hàm getTotalBytesRead**: trả về byte tích lũy

```
def getTotalBytesRead(self):
    """Get total bytes read from the video file."""
    return self.totalBytesRead
```

- **Hàm reset**: reset stream về ban đầu

```
def reset(self):
    """Reset the video stream to the beginning."""
    self.file.seek(0)
    self.frameNum = 0
    self.totalBytesRead = 0
    self.lastFrameSize = 0
```

- **Hàm detectFormat**

```python
def _detectFormat(self):
    """
    Auto-detect video format.

    Supports:
    - Custom format: 5-byte decimal length prefix
    - Standard MJPEG: JPEG markers (FF D8 ... FF D9)
    """
    pos = self.file.tell()
    firstBytes = self.file.read(5)
    self.file.seek(pos)  # Reset position

    if not firstBytes or len(firstBytes) < 5:
        return 'unknown'

    # Try custom format (5-digit length)
    try:
        length = int(firstBytes)
        if 0 < length < 10000000:  # 10MB max
            return 'custom'
    except:
        pass

    # Check JPEG marker
    if firstBytes[0] == 0xFF and firstBytes[1] == 0xD8:
        return 'mjpeg'

    return 'custom'  # Default
```

- **Hàm _nextFrameMJPEG**

```python
def _nextFrameMJPEG(self):
    """
    Read frame from standard MJPEG file.

    JPEG structure:
    - Start: FF D8 (SOI - Start of Image)
    - End: FF D9 (EOI - End of Image)
    """
    # Find start marker
    while True:
        byte = self.file.read(1)
        if not byte:
            return None
        if byte == b'\xff':
            nextByte = self.file.read(1)
            if nextByte == b'\xd8':  # SOI
                frameData = b'\xff\xd8'
                break

    # Read until end marker
    while True:
        byte = self.file.read(1)
        if not byte:
            break
        frameData += byte

        # Safety check
        if len(frameData) > 10000000:  # 10MB
            print("[VideoStream] Frame too large, skipping")
            return None
```

```
            if byte == b'\xff':
                nextByte = self.file.read(1)
                if not nextByte:
                    break
                frameData += nextByte
                if nextByte == b'\xd9':  # EOI
                    break

        self.frameNum += 1
        self.lastFrameSize = len(frameData)

        # Log large frames
        if len(frameData) > 500000:  # >500KB (4K)
            print(f"[VideoStream] 4K Frame {self.frameNum}: "
                  f"{len(frameData)/1024:.1f} KB")
        elif len(frameData) > 100000:  # >100KB (HD)
            print(f"[VideoStream] HD Frame {self.frameNum}: "
                  f"{len(frameData)/1024:.1f} KB")

        return frameData
```

d.RtpPacket.py

- **Hàm isFragmented**: kiểm tra xem packet có phải một phần của fragmented frame hay không

```
def isFragmented(self):
    """Check if this packet is part of a fragmented frame."""
    return self.fragmentHeader is not None
```

- Hàm getFragmentId: lấy fragment identifier

```
def getFragmentId(self):
    """Return fragment ID."""
    if self.fragmentHeader:
        return (self.fragmentHeader[0] << 8) | self.fragmentHeader[1]
    return 0
```

- **Hàm getTotalFragments**: lấy tổng fragments

```python
def getTotalFragments(self):
    """Return total number of fragments."""
    if self.fragmentHeader:
        return self.fragmentHeader[2]
    return 1
```

- **Hàm getFragmentIndex**: lấy vị trí của fragment

```python
def getFragmentIndex(self):
    """Return fragment index."""
    if self.fragmentHeader:
        return self.fragmentHeader[3]
    return 0
```

e. NetworkStats.py

- **Packets**
  - totalPacketsSent: Tổng số RTP packets đã gửi
  - totalPacketsReceived: Tổng số packets nhận được
  - packetsLost: Số packets bị mất
  - totalBytesSent/Received: Lượng dữ liệu truyền tải
- **Frames**
  - framesSent: Số frames video đã gửi
  - framesReceived: Số frames nhận thành công
  - framesLost: Số frames bị mất do fragment loss
- **Fragments**
  - fragmentsSent: Tổng số fragments đã gửi (cho HD video)
  - fragmentsReceived: Số fragments nhận được
- Hàm **recordPacketSent**: ghi nhận packet gửi đi
- Hàm **recordPacketReceived**: ghi nhận packet nhận, tính latency
- Hàm **recordFragmentSent** / **recordFragmentReceived**: Đếm fragments
- Hàm **recordFrameSent** / **recordFrameReceived**: Đếm frames
- Hàm **recordPacketLost** / **recordFrameLost**: ghi nhận loss
- Hàm **getStats**: trả về dictionary chứa tất cả metrics

- Hàm **printStats**: In statistics ra console
- Hàm **getStatsString**: Format ngắn gọn cho GUI display

VIII. CLIENT-SIDE CACHING

1. Đặt vấn đề

- Ban đầu, Client hoạt động theo cơ chế:
- Nhận gói tin: Socket chờ (recv) dữ liệu từ mạng.
- Ghi đĩa: Dữ liệu nhận được ghi xuống ổ cứng thành file tạm thời (cache.jpg).
- Đọc và Hiển thị: Đọc file từ ổ cứng lên RAM và cập nhật giao diện (GUI).
- Gây ra các vấn đề sau:
  - **Hiện tượng Jitter (Giật hình):** Nếu mạng không ổn định (gói tin đến chậm hoặc mất gói), quá trình hiển thị (recv và updateMovie) sẽ bị chặn đứng, khiến video bị khựng lại.
  - **Hiệu năng I/O thấp:** Việc ghi từng khung hình xuống đĩa cứng (file .jpg) rồi đọc lại lên RAM để hiển thị gây lãng phí tài nguyên và làm chậm tốc độ xử lý.
  - **Hiện tượng nghẽn (Blocking):** Hàm recv() là hàm chặn. Nếu mạng không ổn định, toàn bộ giao diện người dùng (GUI) sẽ bị "đơ" (freeze) cho đến khi nhận được gói tin tiếp theo.
- **Giải pháp:** Memory Buffer (RAM) cho việc ghép mảnh và File Cache (Disk) cho việc hiển thị.

```
# Fragment reassembly buffers
self.fragmentBuffer = {}  # {fragment_id: {fragment_index: data}}
self.fragmentMetadata = {}  # {fragment_id: (total_fragments, frame_number)}
```

- fragmentBuffer: Lưu trữ payload của từng mảnh, được truy xuất thông qua fragment_id (định danh của khung hình) và fragment_index (thứ tự của mảnh).
- fragmentMetadata: Lưu trữ thông tin meta để biết khi nào một khung hình đã nhận đủ số mảnh cần thiết.

2. Quy trình xử lí (Logic Flow)

- Quy trình xử lý gói tin được thực hiện trong hàm listenRtp và handleFragmentedPacket như sau:

- **Nhận diện:** Khi nhận gói RTP, kiểm tra cờ báo phân mảnh thông qua rtpPacket.isFragmented().
- **Lưu Cache (Buffering):**
  - Nếu là gói phân mảnh, dữ liệu không được hiển thị ngay mà được đưa vào self.fragmentBuffer.
  - Hệ thống ghi nhận tiến trình nhận mảnh: Received fragment X/Y.
- **Kiểm tra tính toàn vẹn (Completeness Check):**
  - Mỗi khi nhận một mảnh mới, hệ thống kiểm tra: len(self.fragmentBuffer[fragmentId]) == totalFragments.
  - Nếu điều kiện thỏa mãn (đã nhận đủ mảnh), tiến trình **Tái cấu trúc (Reassembly)** được kích hoạt.
- **Tái cấu trúc (Reassembly):**
  - Duyệt tuần tự từ index 0 đến totalFragments - 1.
  - Nối các byte dữ liệu lại thành một chuỗi byte hoàn chỉnh (completeFrame).
- **Dọn dẹp bộ nhớ (Garbage Collection):**
  - Sau khi ghép xong, xóa dữ liệu khỏi buffer để giải phóng RAM.
  - Cơ chế **Timeout**: Hệ thống tự động xóa các mảnh của các frame quá cũ (cách frame hiện tại > 10 ID) để tránh rò rỉ bộ nhớ do các gói tin bị thất lạc vĩnh viễn.

- **Cơ chế File Caching cho Hiển thị**
  - Sau khi frame được ghép hoàn chỉnh, hệ thống sử dụng cơ chế file tạm để chuyển dữ liệu sang giao diện đồ họa (GUI):

```python
def writeFrame(self, data):
    """Write the received frame to a temp image file. Return the image file."""
    cachename = CACHE_FILE_NAME + str(self.sessionId) + CACHE_FILE_EXT
    file = open(cachename, "wb")
    file.write(data)
    file.close()

    return cachename
```

  - Dữ liệu ảnh (JPEG) được ghi xuống ổ cứng dưới dạng file tạm (cache-[sessionID].jpg).

- Thư viện PIL (Pillow) sẽ đọc file này lên để hiển thị vào Label của Tkinter.

3. Kết quả đạt được

- **Hỗ trợ HD Streaming:** Client đã có thể hiển thị video độ phân giải cao (720p/1080p) bằng cách ghép thành công hàng chục mảnh
  - Ví dụ: ~36 mảnh cho 720p, ~87 mảnh cho 1080p.
- **Khả năng chịu lỗi:**
  - Nếu thiếu một mảnh (fragment loss), toàn bộ frame sẽ bị hủy bỏ (discard) thay vì hiển thị frame bị lỗi (glitch), đảm bảo trải nghiệm nhìn tốt hơn.
  - Việc loại bỏ các gói tin đến muộn (discarded late packet) giúp video không bị "giật lùi" về quá khứ.
- **Giám sát hiệu năng:** Tích hợp NetworkStats cho phép theo dõi thời gian thực số lượng mảnh đã nhận (fragmentsReceived) và tỷ lệ mất gói.

4. Kết luận và hướng phát triển

- Hệ thống Client-side đã hoàn thành tốt việc xử lý phân mảnh và tái cấu trúc gói tin (Reassembly). Đây là bước quan trọng nhất để hỗ trợ streaming video chất lượng cao trên nền tảng UDP.

IX. TÀI LIỆU THAM KHẢO

Lập trình Socket cơ bản:

https://realpython.com/python-sockets/

Đa luồng trong Python:

https://stackoverflow.com/questions/68425239/how-to-handle-multithreading-with-sockets-in-python

Truyền Video từ Server đến Client sử dụng Giao thức UDP Python:

Truyền Video Từ Server Đến Client Sử Dụng Giao Thức UDP PYTHON

Live Video stream using python socket:

Live video stream using python socket programming

Các tài liệu thực hành được đăng tải trên Moodle của môn Mạng máy tính 24CTT2 – Khoa Công nghệ Thông tin – Trường Đại học Khoa học Tự nhiên – ĐHQG TPHCM.